UNIVERSIDAD PERUANA UNIÓN

FACULTAD DE INGENIERÍA Y ARQUITECTURA

E. P. DE INGENIERÍA DE SISTEMAS



FRONTEND PHARMASOFT

CURSO:
Lenguaje de programación II

DOCENTE:
Ing. Benjamín David Reyna Barreto

ESTUDIANTE:
Mishelle Esmeralda Osorio Huashuayo

LIMA, OCTUBRE 2026

C1. Resumen de resultados

Se ejecutaron los 24 casos registrados en la matriz: los 16 casos base, cuatro casos propios de grupos distintos y B-01 a B-04. En la ejecución final, los 24 pasan y no quedan casos fallidos.

Grupo	Ejecutados	Pasan	Fallan
Login	5	5	0
Sesión	5	5	0
Interceptor	3	3	0
Guards	3	3	0
Menú y acciones	3	3	0
401 / 403	1	1	0
Aviso de sesión	4	4	0
Total	**24**	**24**	**0**

C2. Fichas de hallazgos

No se registraron casos fallidos en la ejecución final, por lo que no corresponde abrir fichas de hallazgos pendientes. En L-04, el primer intento terminó en una ruta local inexistente; se corrigió la navegación de retorno y se repitió la comprobación. La evidencia final muestra el regreso a /inicio, sin redirección a un dominio externo (p. 8).

C3. Preguntas de análisis

1. ¿Por qué el backend debe validar cada permiso aunque la interfaz oculte opciones?
El menú, las directivas y los guards ordenan la experiencia de uso, pero se ejecutan en el navegador y pueden manipularse o evitarse enviando la petición directamente. Por eso el servidor debe comprobar la autoridad en cada operación antes de consultar o modificar datos. En E-01 se forzó «Dar de baja» con VENDEDOR: la API respondió 403, la SPA informó la falta de permiso y la sesión siguió activa. La regla de catálogo está protegida en SecurityConfig mediante CATALOGO_ESCRIBIR; esta comprobación del backend es la que impide la operación.

2. ¿Qué ventajas y riesgos tiene sessionStorage frente a localStorage y una cookie HttpOnly? ¿Cuál elegiría para la botica?
sessionStorage limita la persistencia a la pestaña y se elimina al cerrar esa pestaña, aunque un script ejecutado por XSS todavía puede leer el token. localStorage sobrevive al cierre del navegador y también es accesible desde JavaScript, por lo que el token queda expuesto durante más tiempo. Una cookie HttpOnly no puede leerse desde JavaScript y, con Secure y SameSite, reduce el riesgo de robo del token; requiere diseñar protección CSRF. Para una botica en producción elegiría una cookie HttpOnly, Secure y SameSite con defensa CSRF, si el backend puede emitirla. sessionStorage es razonable para esta SPA y su sesión corta, pero no ofrece esa protección frente a XSS.

3. ¿Por qué el interceptor cierra la sesión con 401 y no con 403?
401 indica que la autenticación falta, venció o no es válida, así que la SPA limpia la sesión y pide iniciar sesión de nuevo. 403 significa que el usuario sí está autenticado, pero carece del permiso para esa acción; debe mostrarse el rechazo y conservar su sesión. Si la aplicación cerrara la sesión ante un 403, expulsaría a usuarios válidos por intentar una operación no autorizada y podría hacerles perder el trabajo en curso. El errorInterceptor aplica esta distinción y E-01 confirma que la sesión permanece después del 403.

4. ¿Qué es una redirección abierta y cómo se evita en L-04?
Una redirección abierta ocurre cuando el parámetro de retorno acepta una dirección elegida por quien construye el enlace y la aplicación envía allí al usuario después del login. Puede usarse para llevarlo a una página externa engañosa. En login.ts, safeReturnUrl acepta solo rutas locales que empiezan con una barra simple, rechaza // y barras invertidas y verifica que el origen coincida con el de la SPA; si no es seguro, usa /inicio. En L-04 se comprobó que el login no sale a sitio-falso.com y termina en Inicio.